

Bug Title

1.[High] Social Media Links Are Not Redirecting to Their Respective Social Media Pages

Project

She Guides Me — Social Media / Footer / Contact Us

Bug Summary

The **Facebook, LinkedIn, Instagram, YouTube, and TikTok** links displayed on the She Guides Me website appear as social-media links, but they do not properly redirect users to their respective social-media pages when clicked.

The issue is visible on the mobile layout, particularly in the “**Contact us**” section and social-media section in the footer.

Environment

Field	Details
Application	She Guides Me
Module	Footer / Contact Us
Feature	Social Media Links
Device	Mobile
Browser	Chrome
Testing Type	Functional / UI / Navigation Testing
Severity	Medium
Priority	High
Status	Open

Steps to Reproduce

1. Open the **She Guides Me** website on a mobile device.
2. Navigate to the bottom of the page.
3. Locate the **Contact us** or footer section.
4. Find the social-media links:
 - Facebook
 - LinkedIn

- Instagram
 - YouTube
 - TikTok
1. Click each social-media link.
 2. Observe the resulting behavior.

Expected Result

Each social-media link should:

- Be clickable.
- Redirect to the **correct official She Guides Me social-media profile**.
- Open the social-media page correctly, preferably in a new browser tab/window.
- Use a valid HTTPS URL.
- Not redirect to an incorrect, broken, or unrelated page.

Actual Result

The social-media links are displayed on the page, but clicking them **does not correctly navigate to the intended social-media pages**.

This creates confusion because users expect these links to provide direct access to She Guides Me's social-media profiles.

Recommended Solution

1. Verify the href URLs

Check the HTML implementation of every social-media link.

Example:

```
<a href=https://www.facebook.com/sheguidesme  
target="_blank"  
rel="noopener noreferrer">
```

Facebook

```
</a>
```

Similarly, configure valid URLs for:

```
<a href="https://www.linkedin.com/..." target="_blank" rel="noopener noreferrer">
```

LinkedIn

```
</a>
```

```
<a href="https://www.instagram.com/..." target="_blank" rel="noopener noreferrer">
```

Instagram

```
</a>
```

```
<a href="https://www.youtube.com/..." target="_blank" rel="noopener noreferrer">
```

YouTube

```
</a>
```

```
<a href="https://www.tiktok.com/..." target="_blank" rel="noopener noreferrer">
```

TikTok

```
</a>
```

Use the actual official She Guides Me profile URLs, rather than placeholder or incomplete URLs.

2. Check for Empty or Incorrect Links

Inspect each element and verify that it does not contain:

```
href="#"
```

or:

```
href=""
```

or an incorrect internal route such as:

```
href="/facebook"
```

unless that route is intentionally implemented.

3. Make the Entire Social-Media Item Clickable

On mobile, make sure the icon **and the text** are inside the same anchor:

```
<a href="SOCIAL_MEDIA_URL"
```

```
target="_blank"
```

```
rel="noopener noreferrer"
```

```
class="social-link">
```

```

```

```
<span>Facebook</span>
```

```
</a>
```

This improves usability and makes the clickable area easier to tap.

4. Verify Mobile Responsiveness

Test the links on:

- Chrome Android
- Safari iOS
- Chrome responsive/mobile viewport
- Different screen sizes

Make sure no overlay, container, or CSS pointer-events rule prevents tapping the links.

Description:

The Facebook, LinkedIn, Instagram, YouTube, and TikTok links displayed in the She Guides Me footer/Contact Us section are not functioning as expected. When users attempt to click the social-media links on mobile, they do not properly redirect to the corresponding official She Guides Me social-media pages.

Expected Result:

Each social-media link should be clickable and redirect to the correct official social-media profile.

Actual Result:

Social-media links are displayed but do not properly navigate to their intended destinations.

Suggested Fix:

Verify and correct the href attributes for all social-media links, ensure the complete icon/text area is clickable, use target="_blank" with rel="noopener noreferrer" where appropriate, and test the links across mobile and desktop browsers.

Severity: Medium

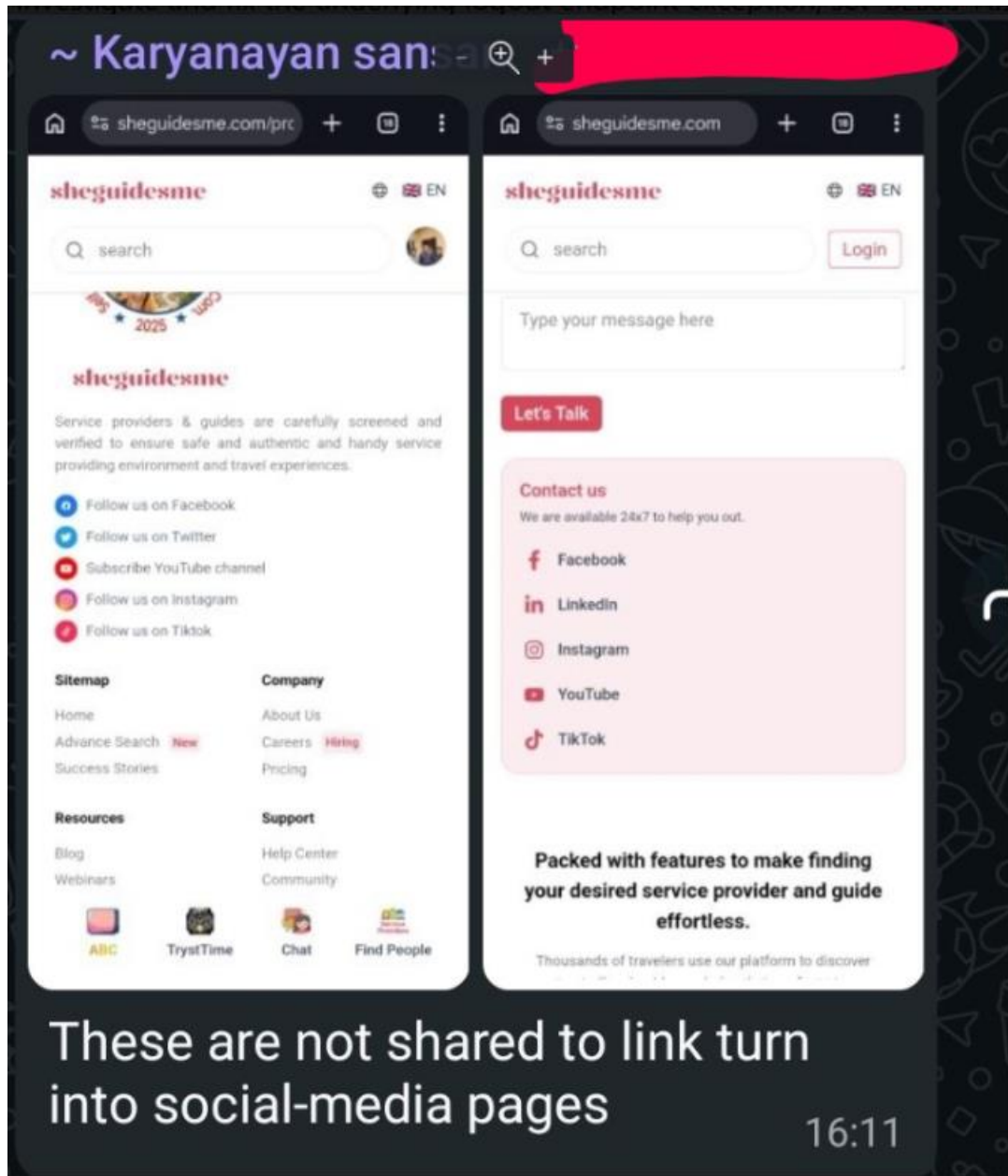
Priority: High

Component: Footer / Contact Us / Social Media

Type: Functional + Navigation + UI/UX

Status: Open

Screenshot:



BUG 3

Title: Typo in section header "Filer by Services" instead of "Filter by Services" and validation triggers before input selection

Bug ID: 3

Issue Type: UI / Spelling / Usability Bug

Module: Search / Filter Modal (/profile/me)

Severity: Minor

Priority: Low

Environment: Web Browser (Desktop)

Description

In the search/filter modal on the profile page

(https://sheguidesme.com/profile/me), **there are two notable issues:**

1. **Spelling Typo:** The category header displays "Filer by Services" instead of the correct label "Filter by Services".
2. **Premature Validation:** The red error message *"Select at least one filter to search"* displays at the top of the modal before the user has attempted to click the "Search Service Provider" button.

Steps to Reproduce

1. Navigate to https://sheguidesme.com/profile/me
2. Open the search/filter overlay modal.
3. Observe the section heading above the service checkboxes.
4. Observe the validation message displayed at the top of the modal.

Expected Result

- The header should read "Filter by Services".
- Validation errors (*"Select at least one filter to search"*) should only appear after the user clicks Search Service Provider without selecting any criteria, not on initial modal load.

Actual Result

- The heading contains a typo reading "Filer by Services".
- The validation message appears immediately upon modal display.

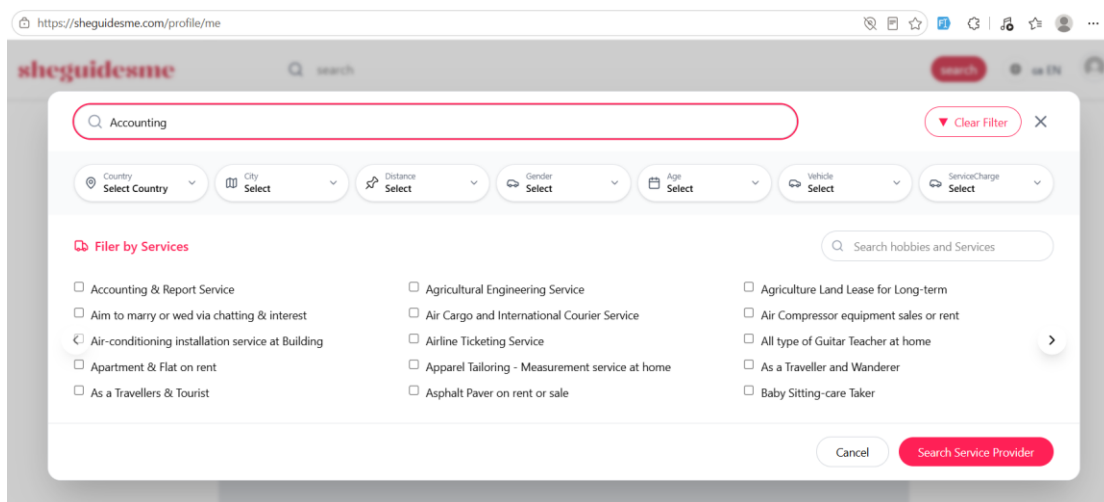
Impact

- Copy & Polish: Minor spelling mistake reduces professional look and feel.
- User Experience: Displaying an error state prematurely can confuse users before they interact with the form.

Suggested Technical Fix

- Update static text string in the modal component template from Filer by Services to Filter by Services.
- Adjust validation state logic so the error banner triggers strictly onFormSubmit rather than initial onMount/onOpen.

Screenshot:



Title: "Unable to fetch location" error pops up automatically on Profile Settings page without user action, and Age defaults to "0"

Bug ID: 4

Issue Type: Functional / UI Bug

Module: Profile Settings (/profile/settings)

Severity: Medium

Priority: Medium

Environment: Web Browser (Desktop)

Description:

(https://sheguidesme.com/profile/settings), a toast notification error displaying "Unable to fetch location" appears immediately without the user toggling the "Use my current location" switch. Additionally, the **Age** field displays a default value of 0 before any Date of Birth is selected, which is logically invalid.

Steps to Reproduce

1. Navigate to
https://sheguidesme.com/profile/settings (<https://sheguidesme.com/profile/settings>).
2. Observe the top-right corner of the viewport immediately upon page load.
3. Observe the **Age** input field under the settings form.

Expected Result

- The geolocation fetch should only trigger when the user explicitly clicks/enables the **"Use my current location"** toggle switch. No error toast should display on page load.
- The **Age** field should remain blank or disabled until a valid **Date of Birth** is selected, or calculate dynamically without defaulting to 0.

Actual Result

- A red toast alert reading *"Unable to fetch location"* displays automatically on initial page load.
- The **Age** field defaults to 0.

Impact

- **User Experience:** Immediate error popups on page load degrade user trust and create confusion regarding browser permissions.
- **Data Integrity:** Defaulting age to 0 can result in invalid form submissions if left unedited.

Suggested Technical Fix

- **LocationLogic:** Ensure the browser Geolocation API (`navigator.geolocation.getCurrentPosition`) is bound exclusively to the `onChange` / `onClick` handler of the location toggle switch, rather than running inside an initial `useEffect` or `componentDidMount` hook.
- **Age Calculation Logic:** Set the default state of the Age field to empty ("" or null) when Date of Birth is empty, updating it dynamically only when a valid date is picked.

Suggestions & UX/UI Feedback

- **Location Permission Context:** Before invoking `navigator.geolocation`, consider showing a lightweight inline explanation or confirmation prompt so users understand *why* location access is being requested.

- **Age Field Automation:** Since **Date of Birth** uniquely determines **Age**, make the Age input strictly read-only and automatically compute the value once a DOB is selected, eliminating manual entry errors.
- **Form Spacing & Alignment:** The bottom "Next" button aligns to the left outside the white card container boundary. Aligning it inside the form card or inline with the right edge will create a cleaner, more balanced UI layout.

Screenshot:

The screenshot shows a web browser window with the URL <https://sheguidesme.com/profile/settings>. The page features a search bar and a navigation menu. The main content area contains a form with the following fields:

- Contact Number:** A text input field with a dropdown menu showing '+977'.
- Alternate Phone/Cell (optional):** A text input field with the placeholder text 'Enter your phone/social media number'.
- Exact City Name:** A text input field with the placeholder text 'Enter your exact city/village name'.
- Location:** A text input field with a toggle switch labeled 'Use my current location'.
- Date of Birth:** A text input field with the placeholder text 'mm/dd/yyyy' and a calendar icon.
- Age:** A text input field with the value '0'.

A red error message 'Unable to fetch location' is displayed in the top right corner. A red 'Next' button is located at the bottom of the form.

Bug Title

5.[Critical] Profile Page Exposes Internal Django Configuration and Server Error Details

Project

She Guides Me — Profile Page

Environment

- **Website:** [She Guides Me](#)
- **Page:** Profile
- **Browser:** Chrome / Microsoft Edge
- **Testing Type:** Functional + Security Testing
- **Severity:** Critical
- **Priority:** High
- **Status:** Open

Description

When viewing a user's profile page, the application displays a large **red error/debug panel** containing internal Django configuration and server information.

The exposed information includes sensitive application settings such as:

- DATABASE_ROUTERS
- DATA_UPLOAD_MAX_MEMORY_SIZE
- DATA_UPLOAD_MAX_NUMBER_FIELDS
- DATE_FORMAT
- DATETIME_FORMAT
- DATETIME_INPUT_FORMATS
- Database-related configuration
- Django application settings and other internal configuration values

This information should **never be displayed to end users** in a production environment.

The screenshot also shows that the error panel overlaps the profile page and significantly affects the UI.

Steps to Reproduce

1. Open [She Guides Me](#).
2. Navigate to a user profile.
3. Open/view the profile details.
4. Observe the right side of the page.
5. Check the red error/debug panel.
6. Observe that internal Django configuration details are displayed.

Expected Result

- The profile page should load normally without exposing internal application details.
- If an unexpected server-side error occurs, users should receive a **generic error message**, such as:

"Something went wrong. Please try again later."

- Django debug information and configuration values should remain hidden from users.
- The error page should not expose database or application configuration.

Actual Result

A red error/debug panel is displayed on the profile page containing **internal Django settings and server configuration information**.

The error panel also covers part of the page and negatively affects the profile page layout and usability.

Impact

Security Impact

- Internal application configuration is exposed.
- Attackers may obtain information useful for further reconnaissance.
- Database and framework configuration details should not be accessible to unauthorized users.
- Indicates that production error/debug handling may be incorrectly configured.

UI/UX Impact

- The error panel obstructs the profile page.
- The page appears broken to the user.
- Profile information becomes difficult to view.

Suggested Solution

1. Disable Django Debug Mode in production:

DEBUG = False

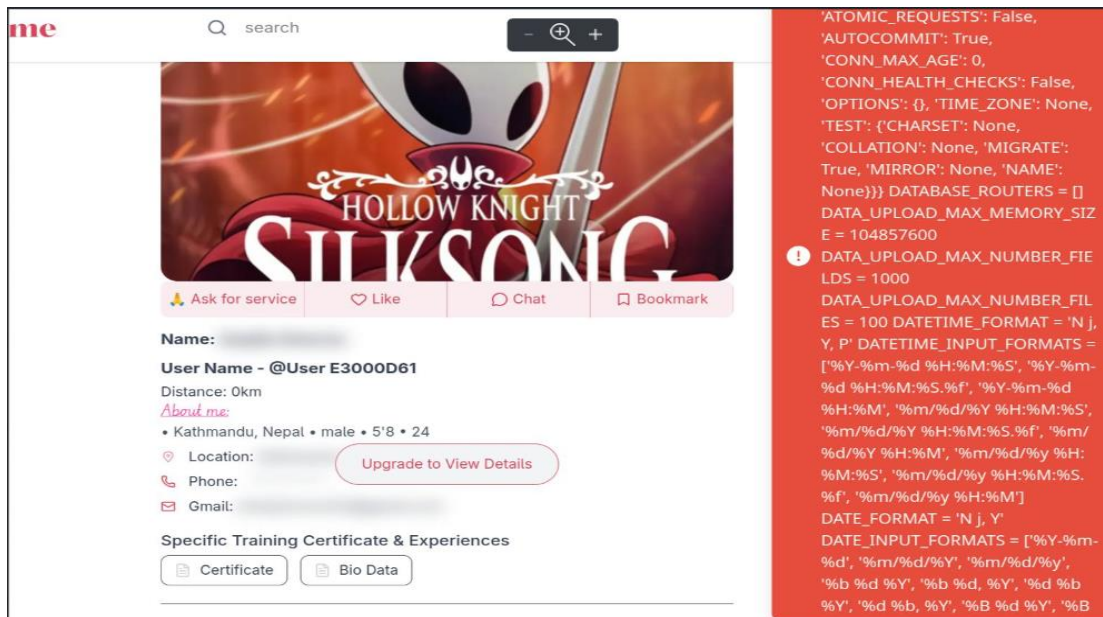
Configure production-safe error handling so that users receive only a generic error page.

2. Review Django production settings and ensure sensitive configuration is never rendered in HTTP responses.
3. Configure proper server-side logging so developers can investigate errors without exposing stack traces or settings to users.
4. Verify that sensitive values such as database configuration, environment variables, and framework settings are not included in frontend responses.
5. Add custom **404/500 error pages** that provide a user-friendly message without technical details.
6. After applying the fix, perform security regression testing on profile pages and other pages that could trigger server-side exceptions.

Acceptance Criteria

- No Django debug/technical information is visible to normal users.
- No database or application configuration is exposed.
- Server errors display a generic user-friendly message.
- Profile page remains visually functional when an error occurs.
- Production environment has DEBUG=False.
- Error details remain available only in secure server-side logs.
- Security regression testing passes.

Evidence



Bug Title

6.[Critical] Logout Button Displays Internal Django Configuration and Server Error Details

Summary

When an authenticated user clicks the **Logout** button, instead of being successfully logged out and redirected to the login/home page, the application displays a large **red error message containing internal Django configuration and environment details**.

The screenshot shows sensitive backend configuration such as `ALLOWED_HOSTS`, `BASE_DIR`, `BASE_URL`, Redis/cache configuration, authentication settings, and other application settings. This indicates that an unhandled server-side exception is being exposed to the client.

Steps to Reproduce

1. Open the **She Guides Me** application.
2. Log in with a valid user account.
3. Navigate to the authenticated user area.
4. Click the **Logout** button.
5. Observe the response displayed by the application.

Expected Result

- User should be successfully logged out.
- User session/authentication token should be invalidated.
- User should be redirected to the **Login page** or public homepage.
- No backend configuration, stack trace, environment variables, or internal application information should be displayed.

Actual Result

- Clicking **Logout** triggers a server-side error.
- A red error panel is displayed.
- Internal Django application configuration is exposed to the browser, including information such as:

ALLOWED_HOSTS

BASE_DIR

BASE_URL

CACHES

AUTHENTICATION_BACKENDS

AUTH_USER_MODEL

Redis configuration

ASGI_APPLICATION

The screenshot also indicates that the application is exposing Django debug/error information to the end user.

Security Impact

This is more than a normal UI/functional bug because the error response exposes **internal server configuration**.

An attacker could potentially use exposed configuration information to:

- Understand the application's backend architecture.
- Identify technologies and services being used.
- Discover internal paths and configuration structure.
- Obtain information useful for further attacks.
- Identify improperly configured production/debug settings.

Recommended severity: Critical, particularly if this is occurring in a production environment.

Recommended Solution

1. Disable Django Debug Mode in Production

Check the production Django settings:

DEBUG = False

Do **not** expose Django's technical debug page to end users.

Use environment variables rather than hardcoding:

```
DEBUG = os.getenv("DEBUG", "False").lower() == "true"
```

For production:

```
DEBUG=False
```

2. Fix the Actual Logout Exception

The most important fix is to identify the exception generated by the logout endpoint. Check the backend/server logs when the logout request is made. For Django, the logout implementation should correctly invalidate the user's session:

```
from django.contrib.auth import logout
```

```
from django.shortcuts import redirect
```

```
def logout_view(request):
```

```
    logout(request)
```

```
    return redirect("login")
```

If you're using Django REST Framework/token authentication, make sure the logout API correctly handles token/session invalidation and returns an appropriate response.

Important: Don't simply hide the error. The underlying logout exception must also be fixed.

3. Return a Safe Error Response

Production users should receive something like:

```
{
  "success": false,
  "message": "An unexpected error occurred. Please try again."
}
```

instead of Django's internal configuration/debug output.

4. Configure a Custom 500 Error Page

For a web application, configure a user-friendly error page:

```
# urls.py
```

```
handler500 = "app.views.server_error"
```

Example:

```
from django.shortcuts import render
```

```
def server_error(request):
```

```
    return render(request, "500.html", status=500)
```

The 500.html page should contain only a generic message such as:

Something went wrong. Please try again later.

It should **never display Django settings or stack traces**.

5. Protect Sensitive Configuration

Review the production configuration and ensure secrets are stored in environment variables/secrets management rather than exposed through responses.

For example:

```
SECRET_KEY=*****
```

```
DATABASE_URL=*****
```

```
REDIS_URL=*****
```

```
DEBUG=False
```

Also make sure error responses cannot expose:

- SECRET_KEY
- database credentials
- API keys
- Redis credentials
- authentication configuration
- server filesystem paths
- environment variables

- stack traces

Suggested Regression Test Case

TC ID	Test Scenario	Steps	Expected Result	Status
TC-LOGOUT-001	Verify successful logout	Login → Click Logout	User is logged out and redirected to Login page	✗ Failed
TC-LOGOUT-002	Verify logout does not expose server details	Login → Logout	No Django debug/configuration information is displayed	✗ Failed
TC-LOGOUT-003	Verify session invalidation	Login → Logout → Browser Back	Protected pages should not be accessible	Pending
TC-LOGOUT-004	Verify logout API response	Send logout request	API returns appropriate success response	Pending
TC-LOGOUT-005	Verify production error handling	Trigger logout failure	Generic error is shown; internal details remain hidden	✗ Failed

When a logged-in user clicks the Logout button, the application encounters a server-side error and displays Django internal configuration/debug information to the client instead of completing the logout operation. The response exposes backend configuration such as ALLOWED_HOSTS, BASE_DIR, BASE_URL, authentication settings, Redis/cache configuration, and other internal application details. This creates a security information-disclosure vulnerability and prevents the expected logout workflow.

Expected: User is logged out and redirected to the Login page without exposing backend information.

Actual: Logout generates an error and exposes internal Django configuration/debug information.

Suggested Fix: Investigate and fix the underlying logout endpoint exception, set DEBUG=False in production, configure safe/custom 500 responses, and ensure authentication/session invalidation is handled correctly.

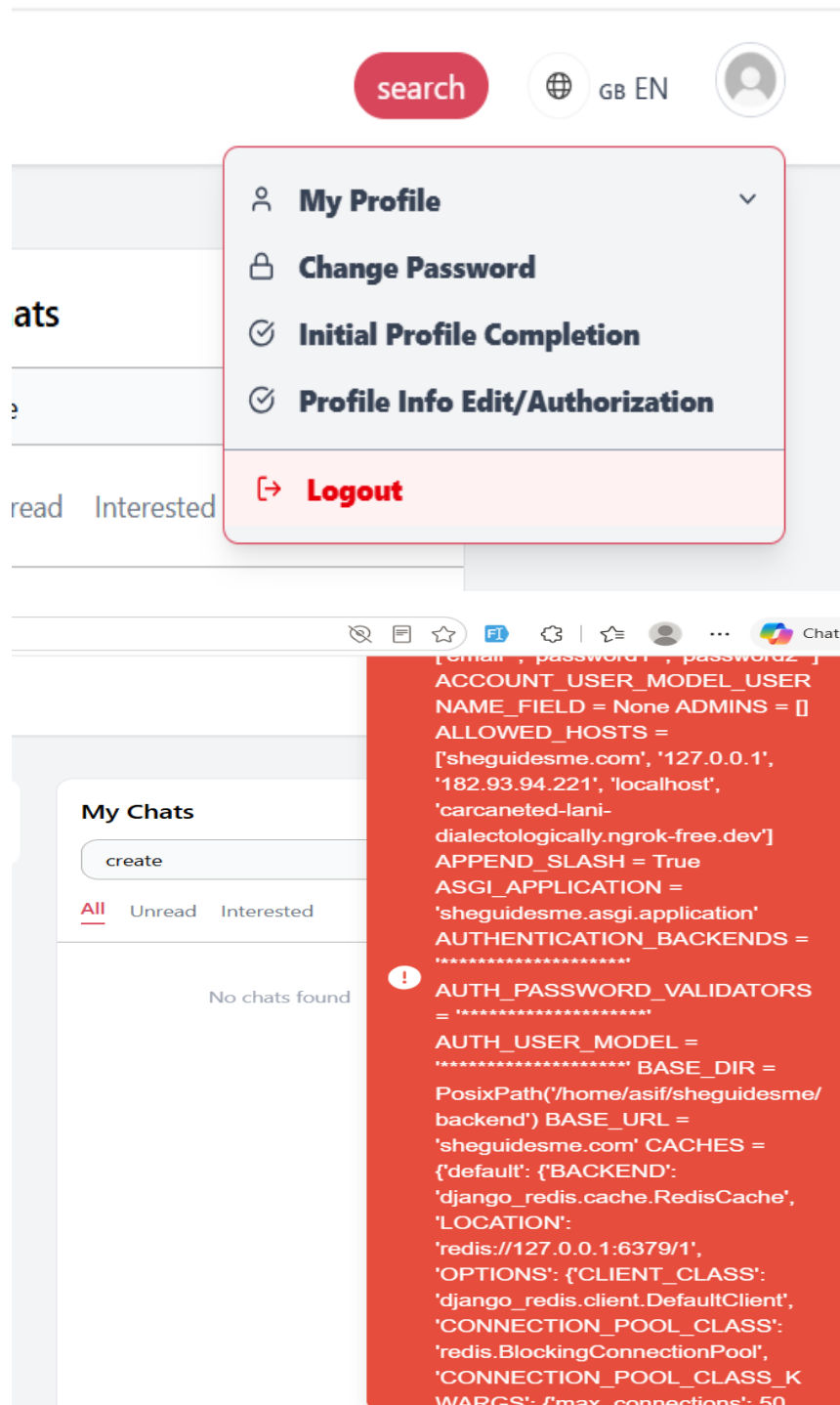
Severity: Critical

Priority: P0 / High

Component: Authentication → Logout

Type: Security + Functional + Information Disclosure

Screenshots:



Bug7 Report: Profile Page Fails to Load Profiles

Bug ID: SGM-PROFILE-001

Project: SheGuidesMe

Module: Profile

Page: <https://sheguidesme.com/profile>

Bug Type: Functional / UI / Data Parsing

Severity: High

Priority: High

Environment: Web Application

Status: New

Summary

The **Profile page fails to load user/service-provider profiles** and displays an **"Error loading profiles"** message with the error **PARSING_ERROR** instead of displaying the available profiles.

Preconditions

- SheGuidesMe website is accessible.
- User navigates to the Profile page.
- Internet connection is available.

Steps to Reproduce

1. Open the SheGuidesMe website.
2. Navigate to the **Profile** page.
3. Wait for the profiles to load.
4. Observe the profile section.

Actual Result

The Profile page does not display any profiles.

Instead, the following error is displayed:

Error loading profiles

Error: PARSING_ERROR

A **Retry** button is also displayed.

Expected Result

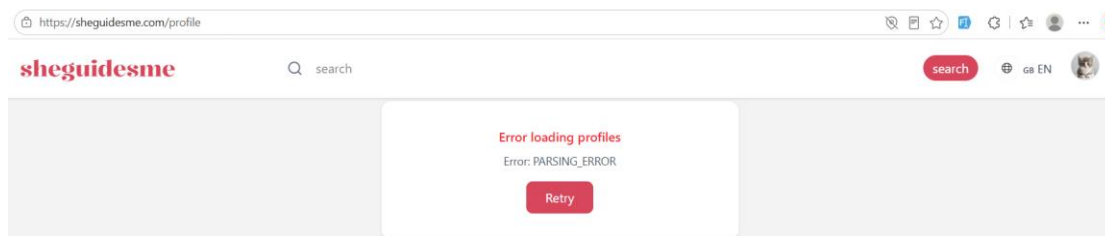
The Profile page should successfully retrieve and display the available profiles without showing a parsing error.

If profile data cannot be retrieved, the application should display a meaningful user-friendly error message and provide an appropriate recovery mechanism.

Impact

- Users cannot view available profiles.
- Users may be unable to discover service providers/guides.
- Profile-related functionality is effectively unavailable.
- This can negatively affect the core user journey of finding verified profiles.

Screenshot Evidence:



Attached Screenshot: Profile page showing Error loading profiles – Error: PARSING_ERROR.

Severity Justification

High: The issue prevents users from accessing profile information, which is a core feature of the platform. The website identifies verified profiles as one of its key features.

Possible Technical Cause

The PARSING_ERROR may indicate that the frontend is receiving profile data in an unexpected format or is unable to parse the response from the profile API. Possible areas to investigate include:

- Invalid or malformed API response.
- Incorrect JSON response structure.
- API response/content-type mismatch.
- Backend serialization issue.
- Unexpected/null profile data.
- Frontend response parsing or schema validation issue.

Recommended Fix

1. Inspect the Profile API request in the browser's **Network** tab.

2. Verify the API response status and response body.
3. Confirm that the response is valid JSON.
4. Compare the API response structure with the frontend's expected schema.
5. Handle null, empty, or malformed profile data gracefully.
6. Add appropriate frontend error handling and logging.
7. Retest the Profile page after the backend/API and frontend fixes.

Acceptance Criteria

- Profile data loads successfully.
- No PARSING_ERROR is displayed during normal operation.
- Profiles are rendered correctly.
- Empty profile results are handled gracefully.
- API failures display a user-friendly error message.
- The **Retry** button successfully retries the profile request.
- Profile functionality works across supported browsers.

Bug 8:

Title: Rejected Works Menu Item Is Partially Clipped in My Profile Sidebar

Expected:

The complete **Rejected** Works menu item should be visible and readable when scrolling through the My Profile sidebar.

Actual:

The **Rejected** Works text is partially hidden/clipped at the bottom of the visible sidebar area, making the option difficult to read or interact with.

Severity: Minor

Priority: Medium

Type: UI / Usability Bug

Module: My Profile

Environment: **Web** – Chrome/Edge/Firefox

Screenshot:

search



GB EN



My Profile



Sanju Sharma

User BC4294B0

94%

Enhance your profile with more details!

[Complete your profile →](#)

[Cancel Membership](#)

Requested Sent

Request Received0

Accepted Works0

Rejected Works0



Change Password



Initial Profile Completion



Profile Info Edit/Authorization



Logout